

SYSTEM NAME: CAMPUS_CARE_LOG

GROUP MEMBERS:

- CHARMAINE GANACIAS – 2024-1538-A

- CATHERINE FERNANDEZ – 2024-1874-A

- RAMEL AZAR – 2024-1388-A

- PEARL KRYSTAL GILLADO – 2024-3921-A

SECTION/BLOCK: BS INFO 2C

COURSE: DATABASE MANAGEMENT SYSTEMS

INSTRUCTOR NAME: KENETTE ROEVEN SAYLO

DATE: JUNE 10, 2026

SYSTEM OVERVIEW

The **campUs cAre Log System** is designed to help school clinics manage patient records more efficiently. It allows nurses to record details about each patient, including personal information, symptoms, diagnosis, treatments, and medicines provided during visits. By storing this data in a structured database, the system ensures that patient histories are easily accessible and organized.

This system is intended for school clinic staff, particularly nurses and administrators, who need a reliable way to track student health records. It solves the problem of manual record-keeping, which can be time-consuming and prone to errors. With features like medicine inventory tracking and visit logs, the system improves accuracy, reduces paperwork, and supports better healthcare services for students.

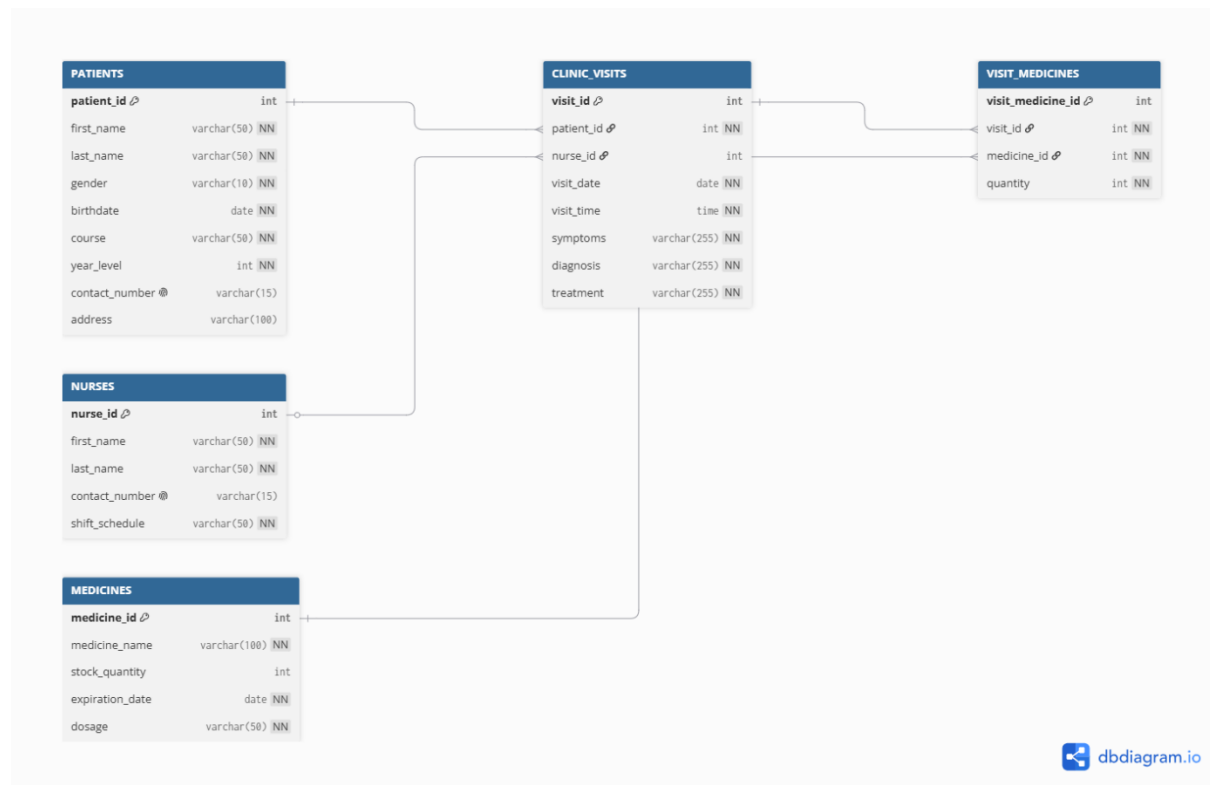
ERD (ENTITY-RELATIONSHIP DIAGRAM)

Notation Used: Crow's Foot

Entities and Attributes:

- **Patients**
 - patient_id (PK)
 - first_name
 - last_name
 - gender
 - birthdate
 - course
 - year_level
 - contact_number
 - address
- **Clinic_Visits**
 - visit_id (PK)
 - patient_id (FK)
 - nurse_id (FK)
 - visit_date
 - visit_time
 - symptoms
 - diagnosis
 - treatment
- **Nurses**
 - nurse_id (PK)
 - first_name
 - last_name
 - contact_number
 - shift_schedule
- **Medicines**
 - medicine_id (PK)
 - medicine_name
 - stock_quantity

- expiration_date
- dosage
- **Visit_Medicines**
 - visit_medicine_id (PK)
 - visit_id (FK)
 - medicine_id (FK)
 - quantity



Relationships:

- One patient can have many clinic visits.
- One nurse can handle many clinic visits.
- One clinic visit can include many medicines.
- One medicine can be used in many clinic visits.
- Visit_Medicines serves as the junction table for the many-to-many relationship between Clinic_Visits and Medicines.

Section 4 — Data Dictionary

campUs cAre Data Dictionary

Table : patients

Table	Field Name	Data Type	Constraints	Description
patients	patient_id	INT	PK, AUTO_INCREMENT	Unique patient ID
patients	first_name	VARCHAR(50)	NOT NULL	Patient first name
patients	last_name	VARCHAR(50)	NOT NULL	Patient last name
patients	gender	VARCHAR(10)	NOT NULL	Gender of patient
patients	birthdate	DATE	NOT NULL	Patient birth date
patients	course	VARCHAR(50)	NOT NULL	Course of patient
patients	year_level	INT	NOT NULL	Current year level of the patient
patients	contact_number	VARCHAR(15)	UNIQUE	Patient contact number
patients	address	VARCHAR(100)	NOT NULL	Home address of the patient

Table : clinic_visits

Table	Field Name	Data Type	Constraints	Description
clinic_visits	visit_id	INT	PK,AUTO_INCREMENT	Unique visit ID
clinic_visits	patient_id	INT	FK	Patient who visited
clinic_visits	nurse_id	INT	FK	Nurse assigned
clinic_visits	visit_date	DATE	NOT NULL	Date of clinic visit
clinic_visits	visit_time	TIME	NOT NULL	Time of clinic visit
clinic_visits	symptoms	VARCHAR(255)	NOT NULL	Symptoms experienced by patients
clinic_visits	diagnosis	VARCHAR(255)	NOT NULL	Nurse or doctor diagnosis
clinic_visits	treatment	VARCHAR(255)	NOT NULL	Treatment given to the patients

Table : visit_medicines

Table	Field Name	Data Type	Constraints	Description
visit_medicines	visit_medicine_id	INT	PK, AUTO_INCREMENT	Unique record ID
visit_medicines	visit_id	INT	FK	Related visit
visit_medicines	medicine_id	INT	FK	Medicine given
visit_medicines	quantity	INT	NOT NULL	Quantity provided

Table : nurses

Table	Field Name	Data Type	Constraints	Description
nurses	nurse_id	INT	PK,AUTO_INCREMENT	unique nurse ID
nurses	first_name	VARCHAR(50)	NOT NULL	Full name of nurse
nurses	last_name	VARCHAR(50)	NOT NULL	Nurse contact number
nurses	contact_number	VARCHAR(15)	NOT NULL	Nurse work
nurses	shift_schedule	VARCHAR(50)	NOT NULL	Work schedule

Table : medicines

Table	Field Name	Data Type	Constraints	Description
medicines	medicine_id	INT	PK,AUTO_INCREMENT	Unique medicine ID
medicines	medicine_name	VARCHAR(100)	NOT NULL	Name of medicine
medicines	stock_quantity	INT	DEFAULT	Available stock
medicines	expiration_date	DATE	NOT NULL	Expiration date
medicines	dosage	VARCHAR(50)	NOT NULL	Dosage Instruction

Section 5 — SQL Script Summary

Stored Procedure

A stored procedure is a saved set of SQL statements that can be reused. It can accept input parameters and perform specific tasks in the database, making code more organized and efficient.

Stored Procedure: Get Complete Visit Details with Patient and Medicine Information

DELIMITER //

```
CREATE PROCEDURE GetVisitDetails(IN visit_id INT)
BEGIN
    SELECT
        cv.visit_id,
        p.first_name AS patient_first_name,
        p.last_name AS patient_last_name,
        p.birthdate AS patient_birthdate,
        p.gender AS patient_gender,
        p.contact_number AS patient_contact,
        n.first_name AS nurse_first_name,
        n.last_name AS nurse_last_name,
        cv.visit_date,
        cv.visit_time,
        cv.symptoms,
        cv.diagnosis,
        cv.treatment,
        m.medicine_name,
        vm.quantity AS medicine_quantity,
        m.dosage AS medicine_dosage
    FROM clinic_visits cv
    INNER JOIN patients p ON cv.patient_id = p.patient_id
    INNER JOIN nurses n ON cv.nurse_id = n.nurse_id
    LEFT JOIN visit_medicines vm ON cv.visit_id = vm.visit_id
    LEFT JOIN medicines m ON vm.medicine_id = m.medicine_id
```

```
WHERE cv.visit_id = visit_id;  
END //
```

DELIMITER ;

How to use it

```
CALL GetVisitDetails(1);
```

Explanation: We created this stored procedure to get all information about a specific clinic visit in one call. It shows the patient's details, the nurse who handled the visit, the diagnosis and treatment, and all medicines given with their quantities. This is useful when I need to display a complete visit report on one page.

View

A view is a virtual table that stores the result of a SQL query. It makes it easier to use complex queries repeatedly without writing them every time. Views improve code organization and security.

View: Patient Visit History Summary

```
CREATE VIEW PatientVisitHistory AS
```

```
SELECT
```

```
    p.patient_id,  
    p.first_name AS patient_first_name,  
    p.last_name AS patient_last_name,  
    p.birthdate,  
    p.gender,  
    p.course,  
    p.contact_number,  
    p.address,  
    cv.visit_id,  
    cv.visit_date,  
    cv.visit_time,  
    cv.symptoms,  
    cv.diagnosis,  
    cv.treatment,  
    n.first_name AS nurse_first_name,  
    n.last_name AS nurse_last_name
```

```
FROM patients p
LEFT JOIN clinic_visits cv ON p.patient_id = cv.patient_id
LEFT JOIN nurses n ON cv.nurse_id = n.nurse_id
ORDER BY p.patient_id ASC, cv.visit_date DESC;
```

How to use it:

```
SELECT * FROM PatientVisitHistory WHERE patient_id = 1;
```

Explanation: We created this view to show the complete visit history for all patients. Instead of writing a complex JOIN query every time I need to see which patients visited the clinic and what happened during their visits, I can simply select from PatientVisitHistory. This view is especially useful for the patient list page where I display each patient's past visits.

JOIN Query

A JOIN query combines rows from two or more tables based on related columns. In my system, I use JOINS to connect patients with their visits, visits with medicines, and nurses with their assigned visits.

JOIN Query: Get All Medicines Given During Clinic Visits with Patient Names

```
SELECT
    cv.visit_id,
    p.first_name AS patient_first_name,
    p.last_name AS patient_last_name,
    cv.visit_date,
    cv.visit_time,
    m.medicine_name,
    vm.quantity,
    m.dosage
FROM clinic_visits cv
INNER JOIN patients p ON cv.patient_id = p.patient_id
INNER JOIN visit_medicines vm ON cv.visit_id = vm.visit_id
INNER JOIN medicines m ON vm.medicine_id = m.medicine_id
ORDER BY cv.visit_date DESC, p.last_name ASC;
```

Explanation: We used this JOIN query in my system to show which medicines were given to which patients during each visit. It connects four tables: clinic_visits (the visit record), patients (to get patient names), visit_medicines (to link visits with medicines),

and medicines (to get medicine names and dosages). This query is helpful for inventory tracking and generating medicine usage reports.

Aggregate Query Example

An aggregate query uses functions like COUNT(), SUM(), AVG(), MAX(), or MIN() to calculate values from groups of rows. These queries help me analyze data and create reports.

Aggregate Query: Count Total Visits and Medicines Given Per Patient

```
SELECT
    p.patient_id,
    p.first_name AS patient_first_name,
    p.last_name AS patient_last_name,
    p.contact_number,
    COUNT(cv.visit_id) AS total_visits,
    COUNT(vm.medicine_id) AS total_medicines_given,
    SUM(vm.quantity) AS total_medicine_quantity
FROM patients p
LEFT JOIN clinic_visits cv ON p.patient_id = cv.patient_id
LEFT JOIN visit_medicines vm ON cv.visit_id = vm.visit_id
GROUP BY p.patient_id, p.first_name, p.last_name, p.contact_number
ORDER BY total_visits DESC;
```

Explanation: I created this aggregate query to analyze which patients visit the clinic most frequently and how many medicines they receive. The COUNT() function counts the number of visits and medicines per patient, while SUM() calculates the total quantity of all medicines given. This query is useful for identifying patients who need regular follow-ups and for inventory planning.

Subquery

A subquery is a query written inside another query. It helps me filter data based on results from another query without needing multiple separate queries.

Subquery: Find Patients Who Have Never Had a Clinic Visit

```
SELECT
    patient_id,
```

```
first_name,  
last_name,  
birthdate,  
gender,  
contact_number,  
address  
FROM patients  
WHERE patient_id NOT IN (  
    SELECT DISTINCT patient_id  
    FROM clinic_visits  
)  
ORDER BY last_name ASC, first_name ASC;
```

Explanation: I used this subquery to find patients in my database who have been registered but have never visited the clinic. The inner query (SELECT DISTINCT patient_id FROM clinic_visits) gets all patient IDs that have at least one visit. The outer query then finds patients whose ID is NOT in that list. This helps me identify inactive patients who might need follow-up calls or reminders.

SQL Queries Used in Our Front-end

We also used these queries in my PHP frontend to connect with the database:

1. Get All Patients for Dropdown (in clinic_visits add form):

```
SELECT patient_id, first_name, last_name  
FROM patients  
ORDER BY first_name ASC;
```

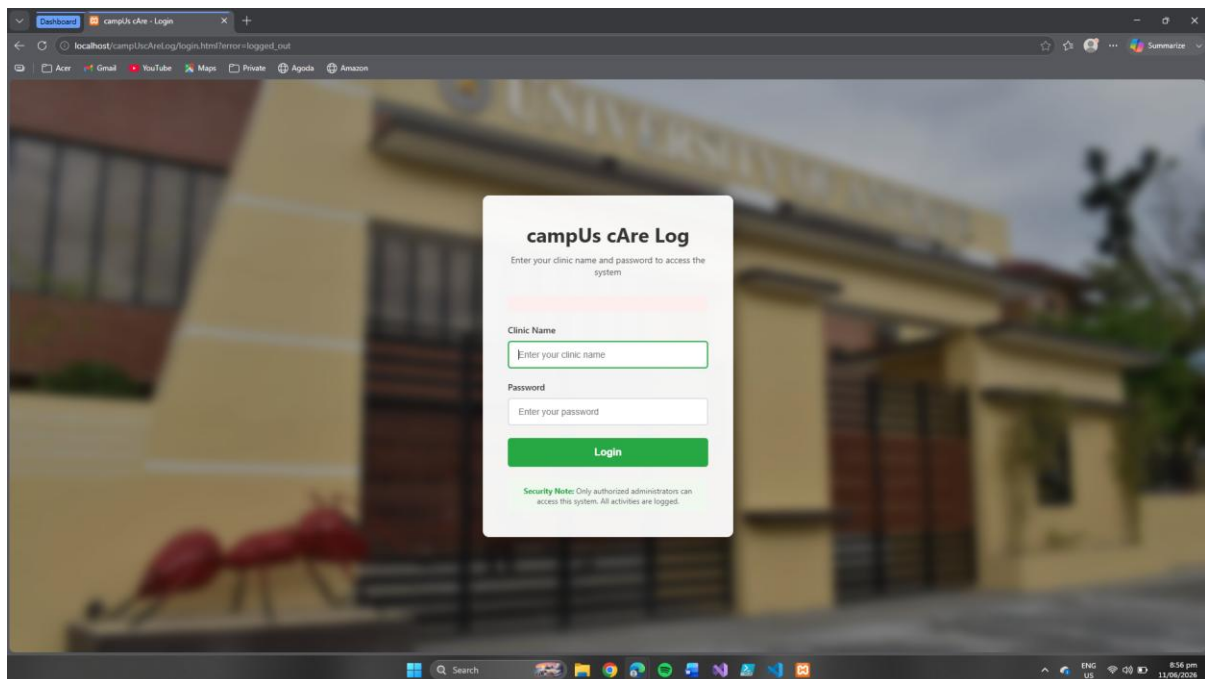
2. Get All Medicines for Dropdown (in visit_medicines add form):

```
SELECT medicine_id, medicine_name  
FROM medicines  
ORDER BY medicine_name ASC;
```

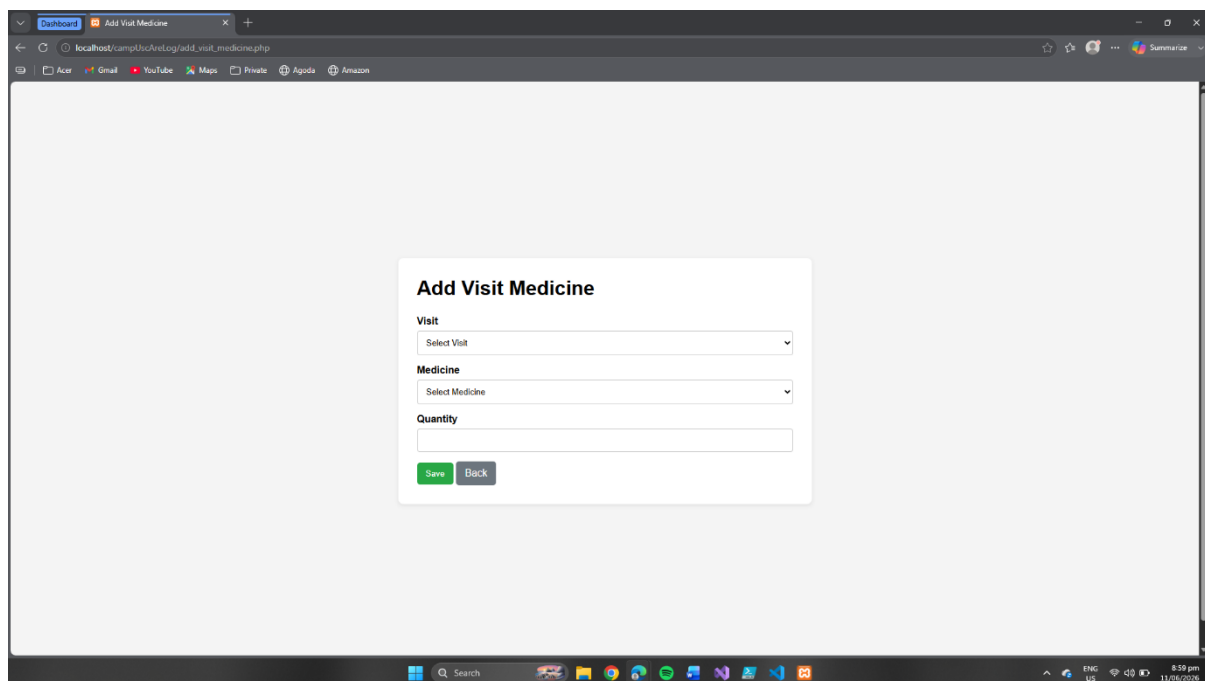
3. Get All Nurses for Dropdown (in clinic_visits add form):

```
SELECT nurse_id, first_name, last_name  
FROM nurses  
ORDER BY first_name ASC;
```

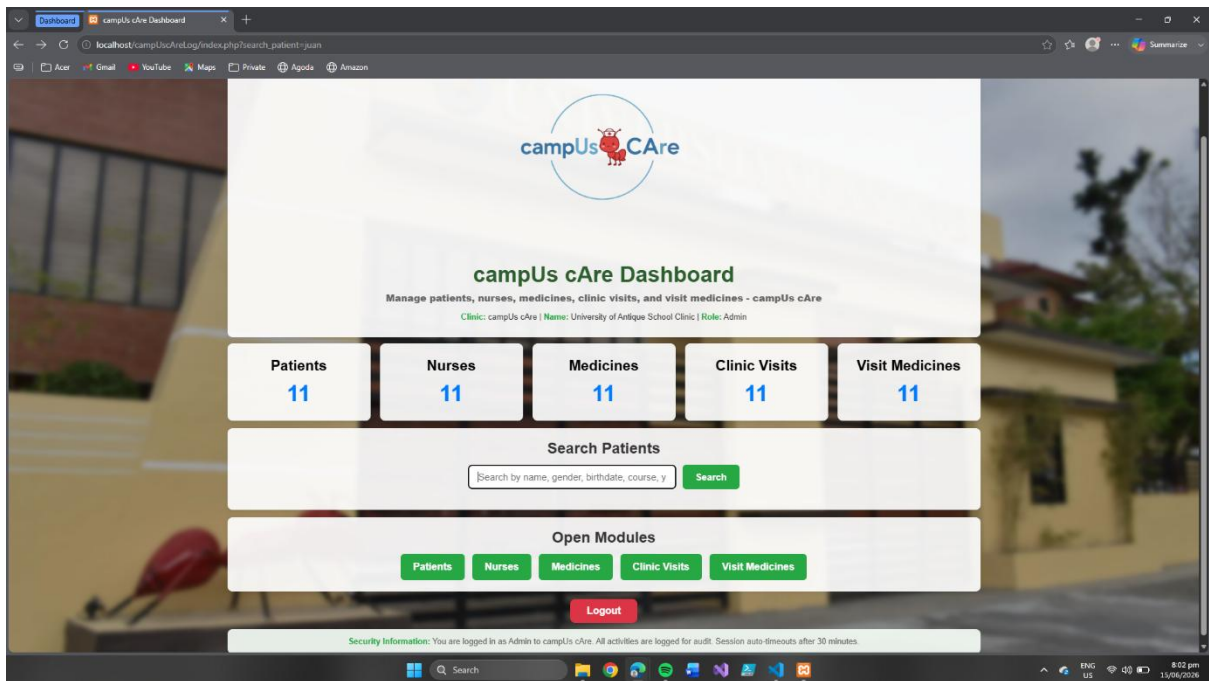
Section 7 — System Screenshots



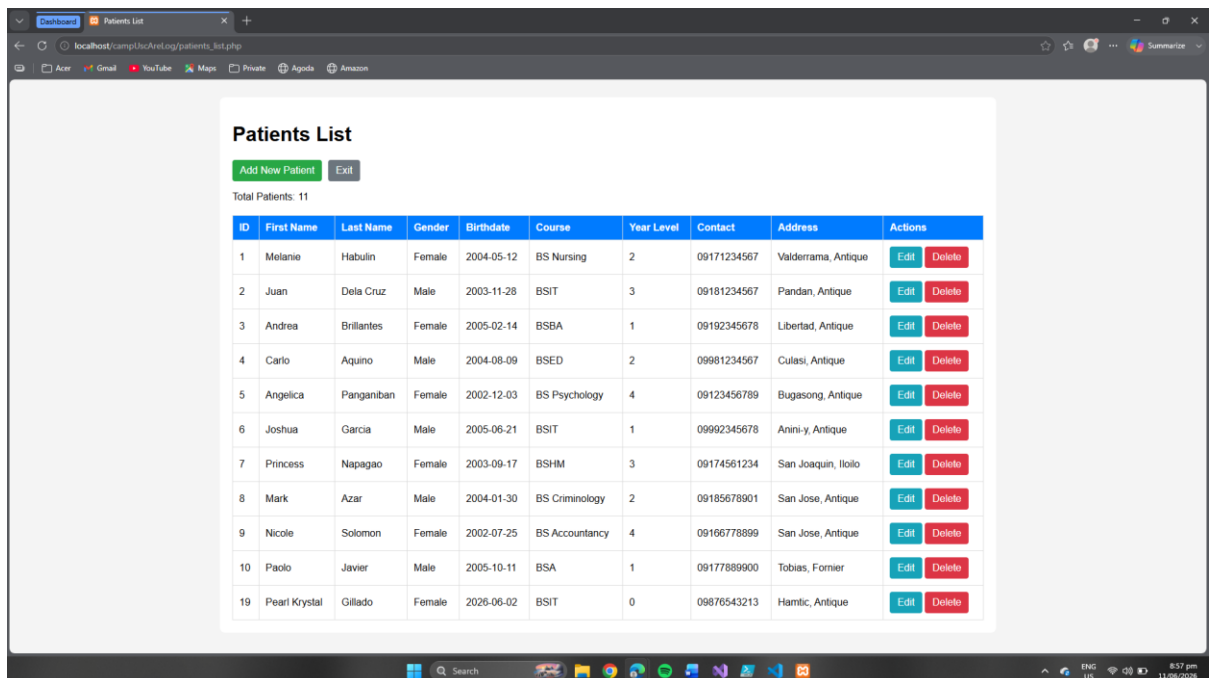
Login Page — Administrators enter clinic name and password to access the system. Security note reminds that only authorized users can log in and all activities are logged.



Add Visit Medicine Page — Administrators select a patient visit, choose medicine, and enter quantity. Provides options to save or go back.



Dashboard Page — Displays clinic overview with counts of patients, nurses, medicines, visits, and visit medicines. Provides quick navigation buttons and logout option, with security information shown at the bottom.



Patients List Page — Displays all registered patients with details such as name, gender, birthdate, course, year level, contact, and address. Provides options to add new patients, edit existing records, or delete entries.

Add Patient

First Name

Last Name

Birthdate

dd/mm/yyyy

Gender

Select Gender

Course

Year Level

Contact Number

Address

Save Back

Add Patient Page — Administrators enter patient details including name, birthdate, gender, course, year level, contact number, and address. Provides options to save or go back.

Nurses List

Add New Nurse Exit

ID	First Name	Last Name	Contact Number	Shift Schedule	Actions
1	Aileen	Gillado	09051230001	Morning Shift	Edit Delete
2	Marjorie	Ganacias	09051230002	Afternoon Shift	Edit Delete
3	Jenelyn	Fernandez	09051230003	Night Shift	Edit Delete
4	Rose	Ysulat	09051230004	Morning Shift	Edit Delete
5	Karen	Seniel	09051230005	Afternoon Shift	Edit Delete
6	Sheryl	Azar	09051230006	Night Shift	Edit Delete
7	Liza	Erispe	09051230007	Morning Shift	Edit Delete
8	Myrna	Vargas	09051230008	Afternoon Shift	Edit Delete
9	Catherine	Sagucom	09051230009	Night Shift	Edit Delete
10	Elena	Napagao	09051230010	Morning Shift	Edit Delete
14	Catherine	Fernandez	09876543213	Night Shift	Edit Delete

Nurses List Page — Displays all registered nurses with contact details and assigned shift schedules. Provides options to add new nurses, edit existing records, or delete entries.

The screenshot shows a web browser window with the address bar displaying 'localhost/camp43cAveLog/add_nurse.php'. The page features a central white form titled 'Add Nurse'. The form contains four input fields: 'First Name', 'Last Name', 'Contact Number', and 'Shift Schedule'. Below these fields are two buttons: a green 'Save' button and a grey 'Back' button. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons. The system clock in the bottom right corner indicates 8:57 pm on 11/06/2026.

Add Nurse

First Name

Last Name

Contact Number

Shift Schedule

Add Nurse Page — Administrators enter nurse details including name, contact number, and shift schedule. Provides options to save or go back.

The screenshot shows a web browser window with the address bar displaying 'localhost/camp43cAveLog/add_medicine.php'. The page features a central white form titled 'Add Medicine'. The form contains four input fields: 'Medicine Name', 'Stock Quantity', 'Expiration Date' (with a date picker icon), and 'Dosage'. Below these fields are two buttons: a green 'Save' button and a grey 'Back' button. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons. The system clock in the bottom right corner indicates 8:58 pm on 11/06/2026.

Add Medicine

Medicine Name

Stock Quantity

Expiration Date

Dosage

Add Medicine Page — Administrators enter medicine details including name, stock quantity, expiration date, and dosage. Provides options to save or go back.

Clinic Visits List

[Add Visit](#) [Exit](#)

ID	Patient	Nurse	Visit Date	Visit Time	Symptoms	Diagnosis	Treatment	Actions
12	Mark Azar	Karen Seniel	2026-06-10	11:04:00	headache	fever	biogesic	Edit Delete
10	Paolo Javier	Elena Napagao	2026-05-24	15:15:00	Weakness and dizziness	Possible dehydration	Fluids and vitamin C	Edit Delete
9	Nicole Solomon	Catherine Sagucom	2026-05-24	10:05:00	Sore throat and fever	Bacterial pharyngitis	Amoxicillin and paracetamol	Edit Delete
8	Mark Azar	Myrna Vargas	2026-05-23	09:25:00	Mild cough	Common cold	Cough syrup	Edit Delete
7	Princess Napagao	Liza Erispe	2026-05-23	08:40:00	Toothache	Dental pain	Mefenamic acid	Edit Delete
6	Joshua Garcia	Sheryl Azar	2026-05-22	14:00:00	Loose bowel movement	Mild dehydration	Oral rehydration salts	Edit Delete
5	Angelica Panganiban	Karen Seniel	2026-05-22	13:20:00	Sneezing and itchy eyes	Allergic rhinitis	Cetirizine	Edit Delete
4	Carlo Aquino	Rose Ysulat	2026-05-21	11:10:00	Body pain after sports	Muscle strain	Ibuprofen and rest	Edit Delete
3	Andrea Brilantes	Jenelyn Fernandez	2026-05-21	10:30:00	Stomach pain	Hyperacidity	Antacid suspension	Edit Delete
2	Juan Dela Cruz	Marjorie Ganacias	2026-05-20	09:00:00	Cough and sore throat	Upper respiratory infection	Amoxicillin and fluids	Edit Delete
1	Melanie Habulin	Aileen Gillado	2026-05-20	08:15:00	Fever and headache	Viral infection	Paracetamol and rest	Edit Delete

Clinic Visits List Page — Displays records of patient visits including patient name, nurse, date, time, symptoms, diagnosis, and treatment. Provides options to add new visits, edit existing records, or delete entries.

Add Clinic Visit

Patient Name

Nurse Name

Visit Date

Visit Time

Symptoms

Diagnosis

Treatment

[Save](#) [Back](#)

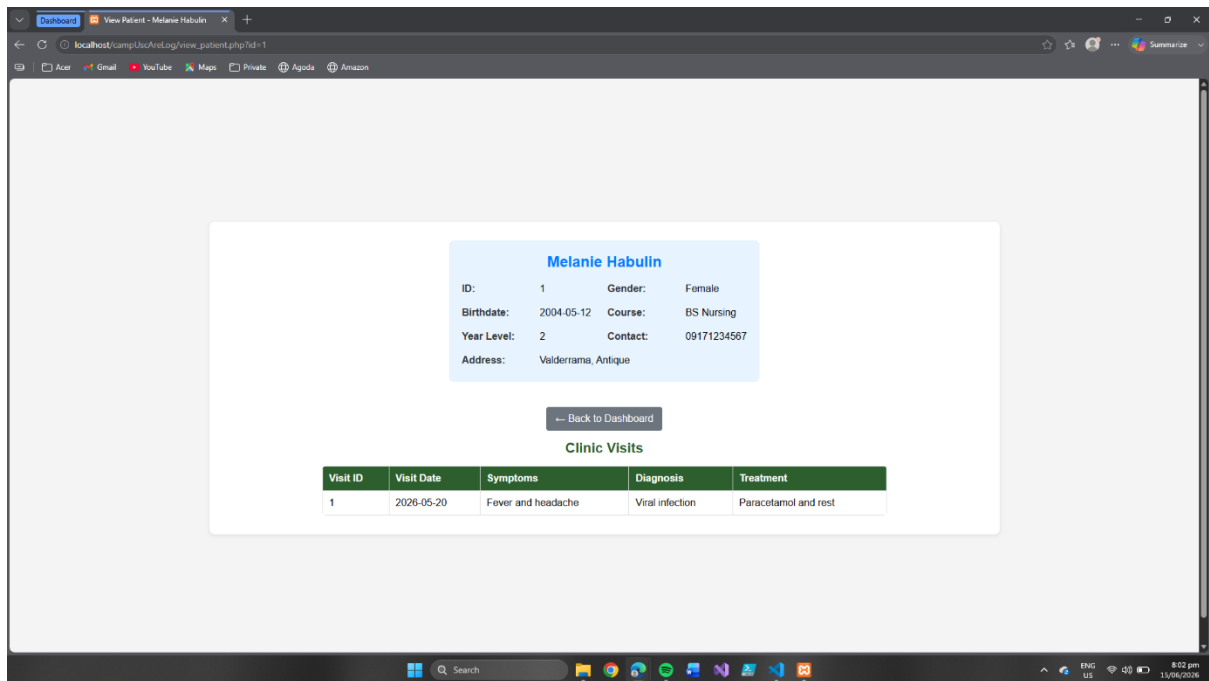
Add Clinic Visit Page — Administrators record patient visit details including patient name, nurse, date, time, symptoms, diagnosis, and treatment. Provides options to save or go back.

ID	Visit ID	Medicine ID	Medicine Name	Visit Date	Quantity	Actions
1	1	1	Paracetamol 500mg	2026-05-20	2	Edit Delete
2	2	2	Amoxicillin 500mg	2026-05-20	3	Edit Delete
3	3	5	Antacid Suspension	2026-05-21	1	Edit Delete
4	4	3	Ibuprofen 200mg	2026-05-21	2	Edit Delete
5	5	4	Cefixime 10mg	2026-05-22	1	Edit Delete
6	6	6	Oral Rehydration Salts	2026-05-22	2	Edit Delete
7	7	7	Mefenamic Acid 500mg	2026-05-23	2	Edit Delete
8	8	9	Cough Syrup	2026-05-23	1	Edit Delete
9	9	2	Amoxicillin 500mg	2026-05-24	2	Edit Delete
10	10	8	Vitamin C 500mg	2026-05-24	1	Edit Delete
14	9	3	Ibuprofen 200mg	2026-05-24	5	Edit Delete

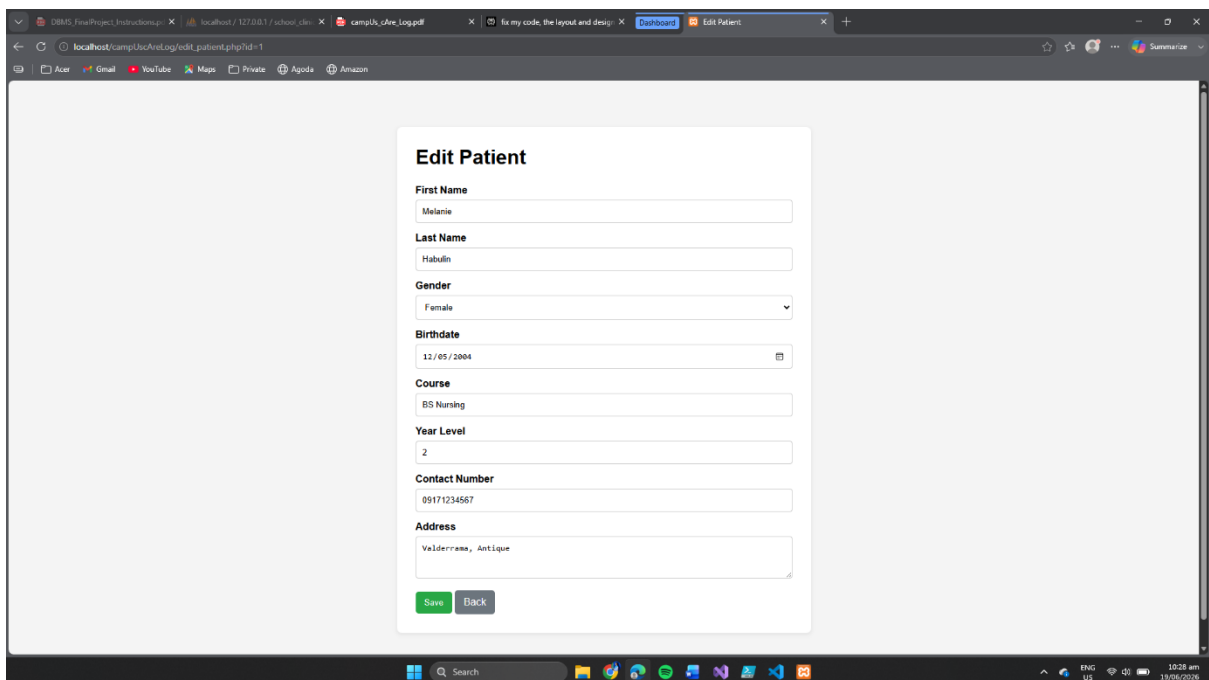
Visit Medicines List Page — Displays medicines prescribed during clinic visits with details such as visit ID, medicine name, date, and quantity. Provides options to add new records, edit existing entries, or delete them.

ID	First Name	Last Name	Gender	Birthdate	Course	Year Level	Contact	Address	Action
1	Melanie	Habulin	Female	2004-05-12	BS Nursing	2	09171234567	Valentiniana, Antique	View

Search Patient Page - Search for patients by name, gender, birthdate, course, year level, contact number, or address to view their details and clinic visits.



When you click "View" on the search page, you'll see the patient's full profile (ID, name, gender, birthdate, course, year level, contact, address) and their complete clinic visit history.



DBMS_FinalProject_Instructions... localhost / 127.0.0.1 / school... campUs_cAre_Log.pdf fix my code, the layout and design: Dashboard Edit Clinic Visit

localhost/campUs_cAre_Log/edit_clinic_visits.php?id=9

Acce Gmail YouTube Maps Private Agoda Amazon

Edit Clinic Visit

Patient
Nicole Solomon

Nurse
Catherine Sagucem

Visit Date
24/09/2026

Visit Time
10:05 am

Symptoms
Sore throat and fever

Diagnosis
Bacterial pharyngitis

Treatment
Amoxicillin and paracetamol

Save Back

Search ENG US 10:29 am 19/06/2026

DBMS_FinalProject_Instructions... localhost / 127.0.0.1 / school... campUs_cAre_Log.pdf fix my code, the layout and design: Dashboard Edit Medicine

localhost/campUs_cAre_Log/edit_medicine.php?id=3

Acce Gmail YouTube Maps Private Agoda Amazon

Edit Medicine

Medicine Name
Ibuprofen 200mg

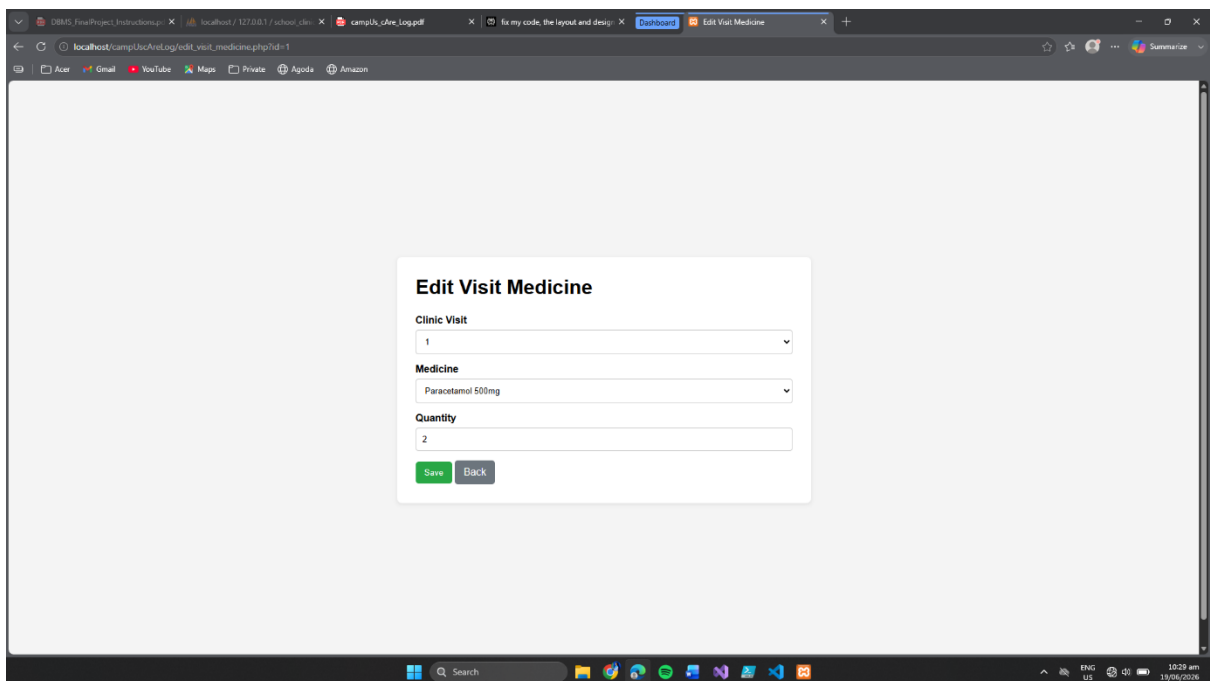
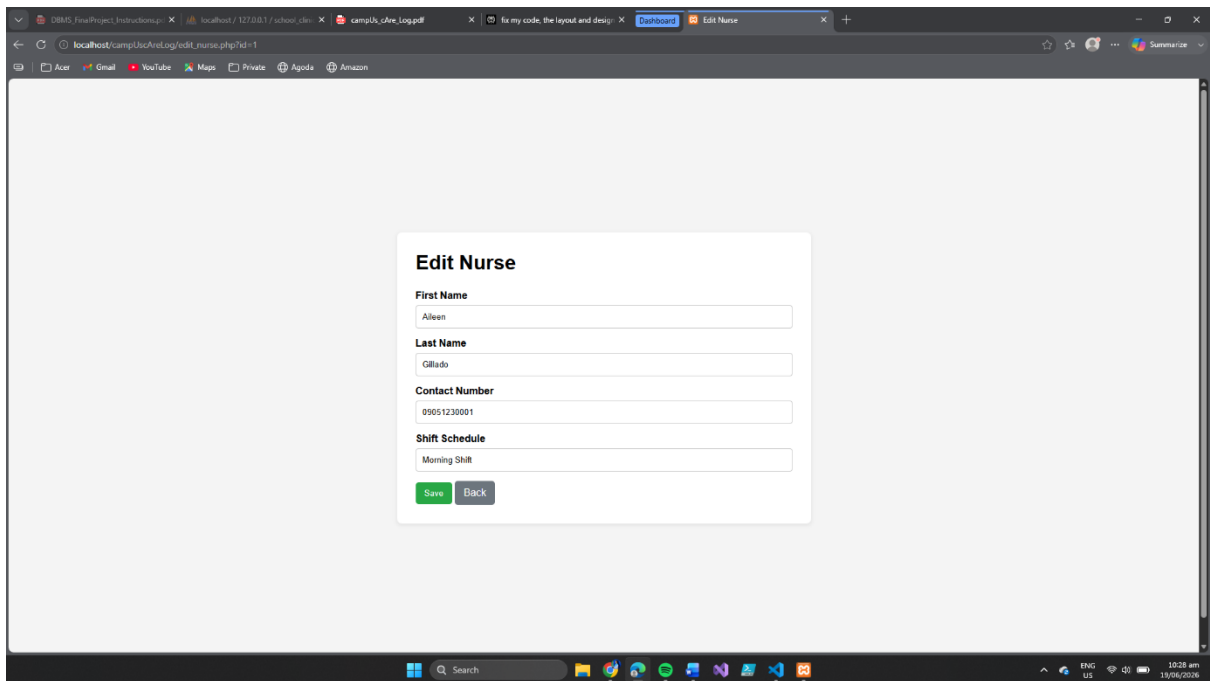
Stock Quantity
60

Expiration Date
28/03/2027

Dosage
1 tablet every 8 hours

Save Back

Search ENG US 10:28 am 19/06/2026



Edit Pages of Each Button

Section 7 — Individual Contribution Summary

CATHERINE FERNANDEZ

I was responsible for creating data dictionary of the campUs-cAre system. I created the patients table to store important student information such as name, gender, birth date, course, year level, contact number, and address. I also created the nurses table to store nurse information, including schedules and contact details. These tables helped organize and manage user information in the system. I also created the medicines table to store medicine details such as the medicine name, stock quantity, expiration date, and dosage instructions. I used the correct data types and rules to keep the information accurate and reliable. This table helps the clinic monitor medicine inventory and manage available supplies. I also added validation rules, primary keys, and required fields to prevent missing or duplicate records. This improved medicine management within the system.

In addition, I created the clinic visits and visit medicines tables. The clinic visits table records patient consultations, including visit dates and times, symptoms, diagnoses, treatments, and the assigned nurse. The visit medicines table records the medicines given during each visit and the quantity provided. I connected these tables using foreign keys to ensure that records were properly linked and maintained. These database components help the system track patient treatments and medicine usage effectively.

Lastly, I reviewed the database design to make sure the data dictionary matched the ERD. This helped reduce confusion during development and provided a clear reference for the group. I carefully checked the documentation to ensure that all fields were well described and easy to understand. My work served as a guide for the team throughout the database development process.

PEARL GILLADO

I contributed by actively supporting my groupmates throughout the completion of our requirements by reviewing their work and providing constructive feedback. Since I did not have access to a laptop, I was unable to participate directly in the technical implementation of the system. However, I made sure to stay involved in the project by carefully checking the outputs that my groupmates produced, including the documentation, screenshots, SQL results, and other required deliverables. I examined whether the information presented was accurate, complete, and aligned with the instructions given by our instructor. Whenever I noticed mistakes, missing details, formatting issues, or areas that could be improved, I shared my suggestions with the group so that necessary revisions could be made before submission.

In addition, I actively participated in group discussions by sharing ideas and opinions regarding the organization of our outputs and the overall presentation of the project. I helped verify that each requirement for each phase was included and properly arranged according to the prescribed format. I also reminded my groupmates about deadlines and encouraged them to review their work thoroughly to avoid errors. Although my contribution was different from those who handled the coding and database implementation, I fulfilled my responsibility by serving as a reviewer and support member of the team. Through continuous communication, attention to detail, and constructive feedback, I helped improve the quality, accuracy, and completeness of our Project submission and contributed to the successful completion of our project as a group.

CHARMAINE GANACIAS

I was responsible for building the entire clinic management system from the very first idea all the way to the final working product that people can use today. This was a big project, and I took care of every single part of it.

First, I created something called an Entity-Relationship Diagram, or ERD for short. Think of this diagram like a big map or a blueprint for the whole system. It showed all the different parts of the clinic, such as patients, medicines, nurses, and clinic visits. The ERD also showed how all these things connect to each other. For example, it showed which medicines a patient gets during a visit, or which nurse handled a certain appointment. This map helped me plan out the entire database before I started building anything, so I knew exactly where everything should go.

Next, I built the actual database using SQL, which is a language for storing and organizing data. I created all the tables that hold the information, like a table for patient records, a table for medicine inventory, a table for nurse information, and a table for clinic visit logs. I made sure each table had the right columns (like name, age, or date) and set up the relationships between the tables, so everything works together properly. This meant that when a doctor records a clinic visit, the system knows which patient it was for and which medicines were given during that visit.

After the database was finished and ready to use, I built the entire frontend of the system. The frontend is everything the user sees and touches on their computer or phone. I created all the web pages and forms that people use to interact with the system. For example, I made a page where you can add a new patient to the system, a page for adding new medicines to the inventory, a page for recording new clinic visits, and a page that lets you link specific medicines to a particular visit. I designed all of these pages to look clean, simple, and consistent. I used the same style, fonts, and colors across the whole system, so it feels organized and easy to use.

Finally, I connected the frontend to the database, so they work together. I used PHP and PDO, which are tools that help web pages talk to databases. This connection makes sure that when someone fills out a form and clicks the "Save" button, the information goes correctly into the database and is stored safely. I also made sure the system can read data from the database and show it on the pages. For example, when you need to pick a patient or a medicine from a dropdown list, the system pulls that list from the database and shows it to you.

In short, I did everything from start to finish. I designed how the data should be organized, built the database to store all the information, created the user interface that people see and use, and made sure the frontend and database work together smoothly. Without any of these parts, the system wouldn't work, and I made sure all of them came together perfectly.

RAMEL AZAR

I gave significant effort to this project and played an important role in its success. I helped create the ERD, carefully checking the tables, keys, and relationships to ensure the system was complete and accurate. I also built my own front-end and back-end design. Even though Charmaine Ganacias' design was ultimately chosen for its smoother performance, my version demonstrated that I could handle both coding and system design effectively.

Beyond development, I was actively involved in brainstorming and decision-making, guiding the team on the right steps to take. I also reviewed the database, identifying missing elements and suggesting improvements to strengthen its structure. Most importantly, I took full responsibility for Phase 5, where I wrote the documentation. This was not just simple writing—I explained the SQL scripts, organized the technical details, and presented the report in a clear and professional way.

Overall, my contribution covered design, coding, database review, team collaboration, and documentation. These combined efforts made me one of the key drivers of the project's success.